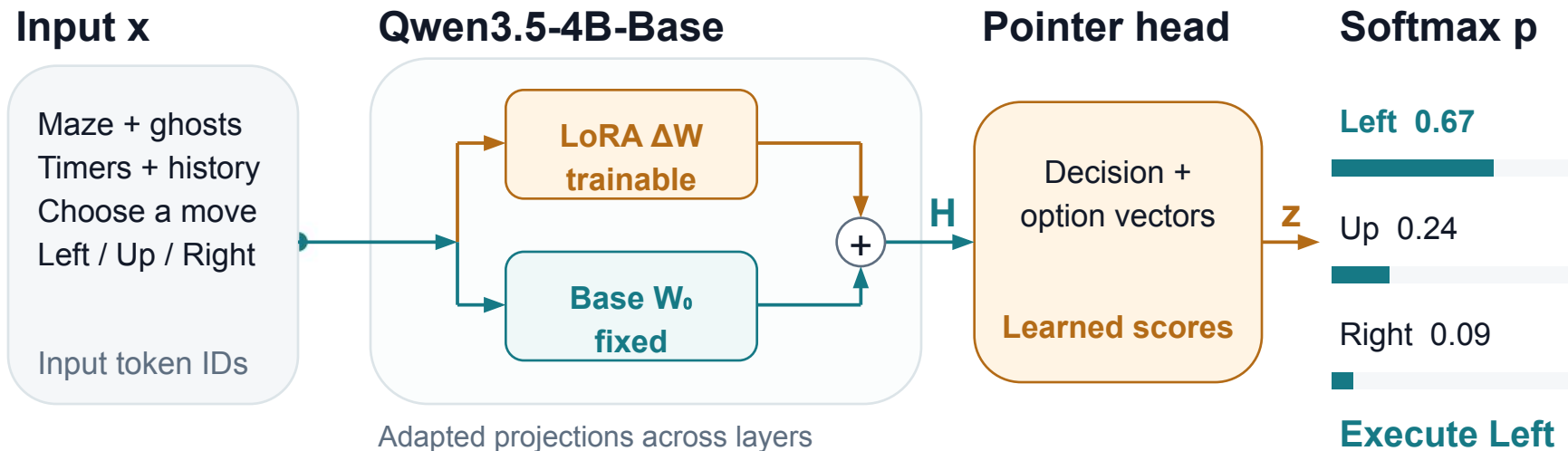


Key architecture

One board, supplied moves, direct probabilities



$$H = f_{\theta}(x)$$

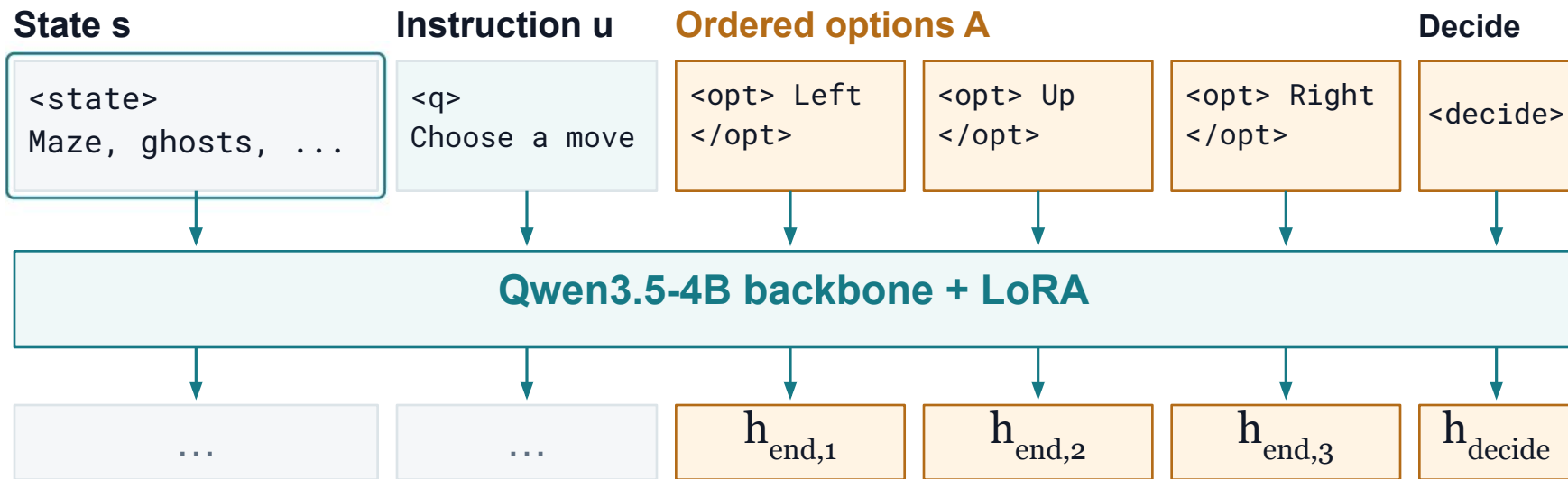
$$p = \text{softmax}(z)$$

x: input tokens H: hidden vectors θ : effective backbone parameters
 W_0 : fixed base weights ΔW : LoRA updates z: scores p: probabilities

Illustrative probabilities. [Open animation controls.](#)

From board state to hidden vectors

State + instruction + legal moves $\rightarrow$ one structured input



The pointer head reads each option ending and the final decision vector

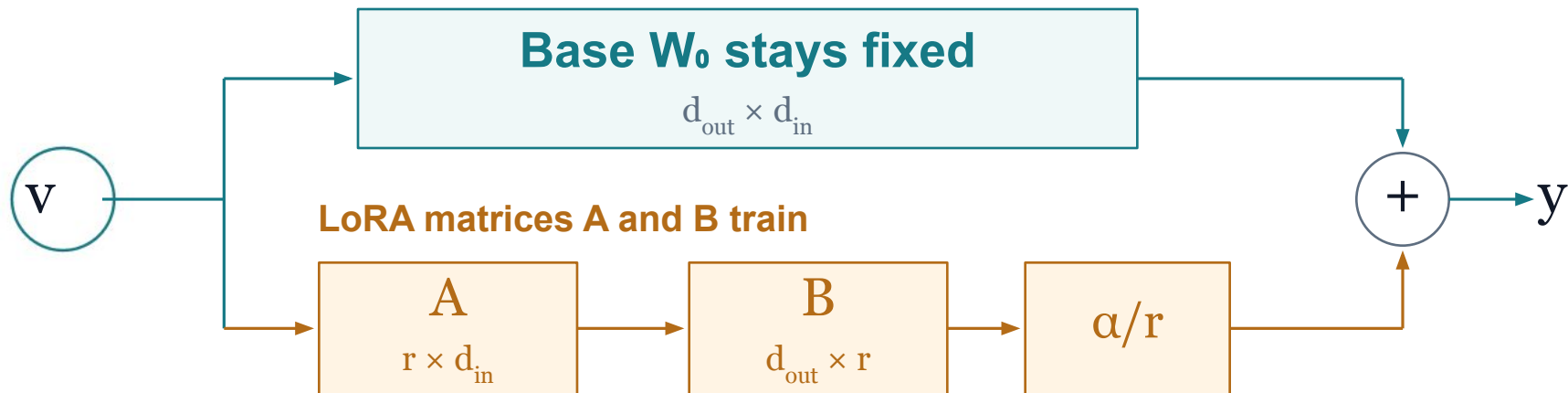
$$\mathbf{x} = \text{encode}(\mathbf{s}, \mathbf{u}, \mathbf{A})$$

$$\mathbf{H} \in \mathbb{R}^{L \times d}$$

L tokens $\times$ d features

LoRA inside Kev

Small learned matrices adapt each selected projection



$$y = W_0 v + (\alpha/r)BAv$$

$$\Delta W = (\alpha/r)BA$$

$r = 16$ $\alpha = 32$ $\alpha/r = 2$ LoRA dropout = 0.05

33.8M trainable parameters across LoRA and the pointer head

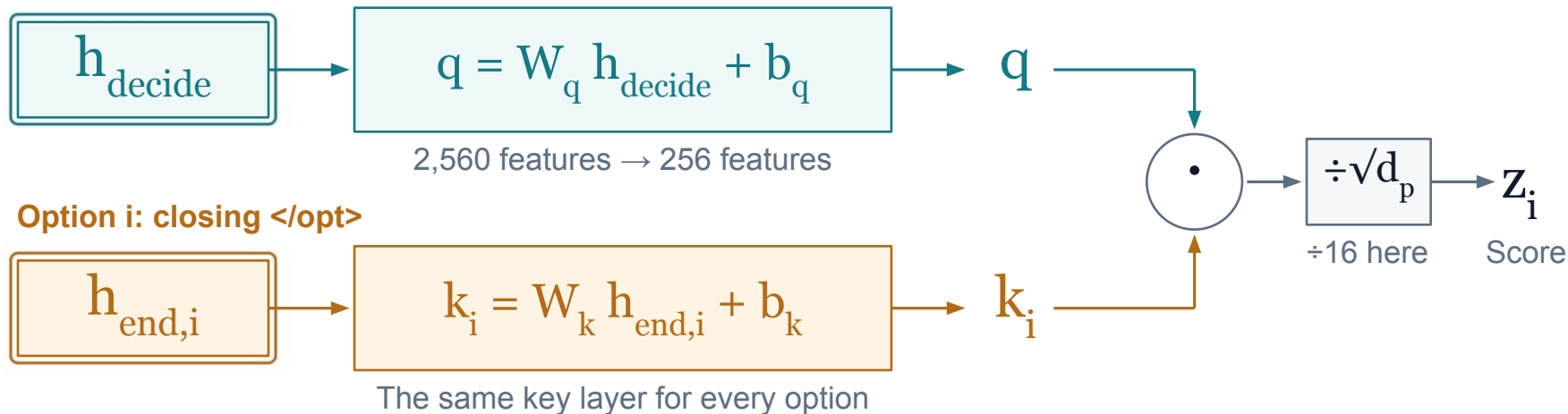
v : layer input y : layer output d_{in} / d_{out} : input / output widths

One adapted projection. Kev uses all targets. [Open animation controls.](#)

The pointer head scores every option

Two learned linear projections, then a scaled dot product

Final <decide> vector



$$z_i = q^T k_i / \sqrt{d_p}$$

K legal options → K raw scores

$d = 2,560$ $d_p = 256$ $\sqrt{d_p} = 16$ Both projection layers train

Softmax and action selection

Softmax gives option probabilities. Argmax chooses the largest.

$$p(a_i | s) = \frac{\exp(z_i / T)}{\sum_{j=1}^K \exp(z_j / T)}$$

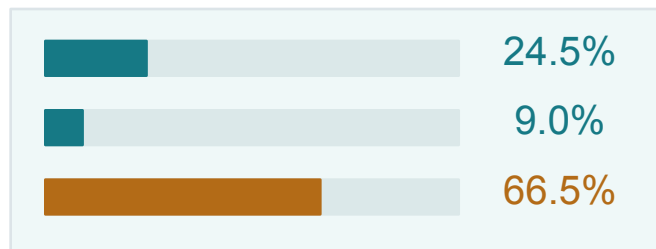
$$T = 1$$

Our fresh lab checkpoint

Raw scores

Left	1.0
Up	0.0
Right	2.0

Option probabilities



Chosen move

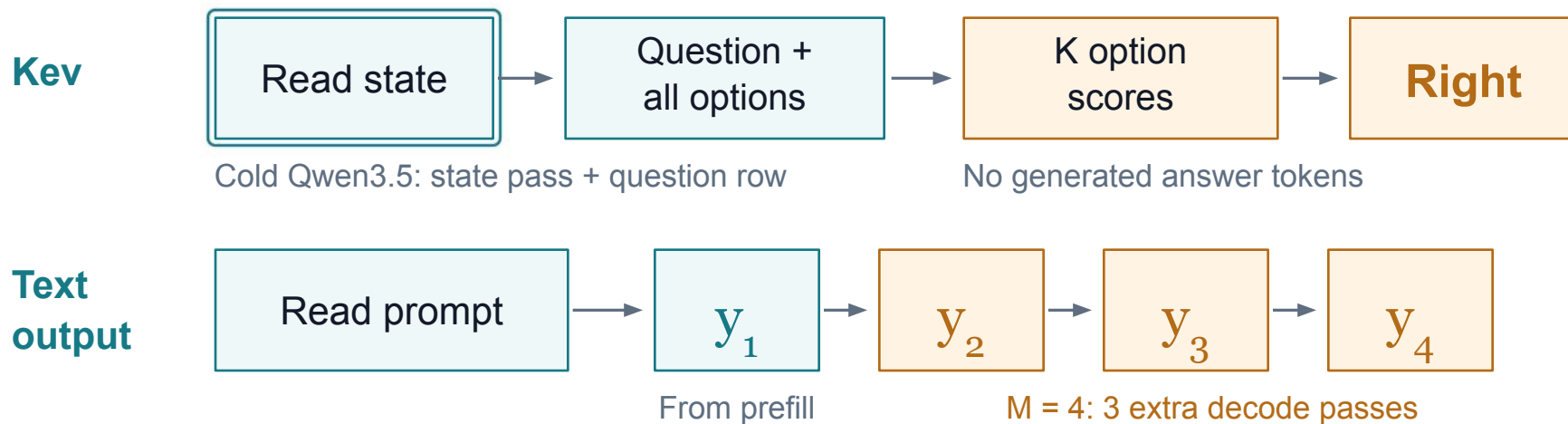
Right

$$i^* = \operatorname{argmax}_i p(a_i | s)$$

$$a^* = a_{i^*}$$

Why Kev avoids the answer-token loop

The backbone reads the input. The pointer head returns K scores.



$$R_{\text{Kev}} \approx (K + 1) d d_p + K d_p$$

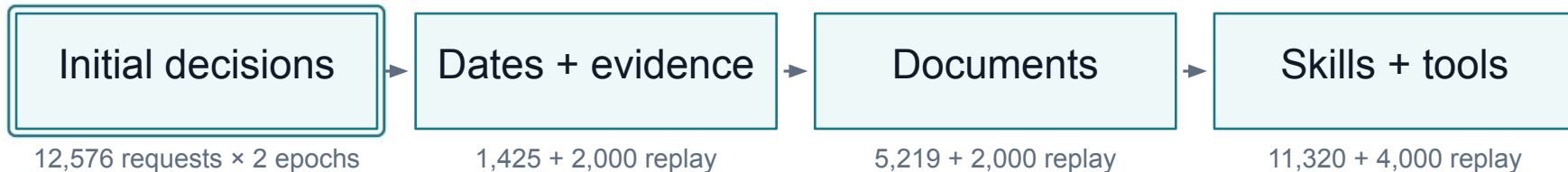
$$R_{\text{text}} \approx M d V$$

Readout multiplications
K choices, V vocabulary

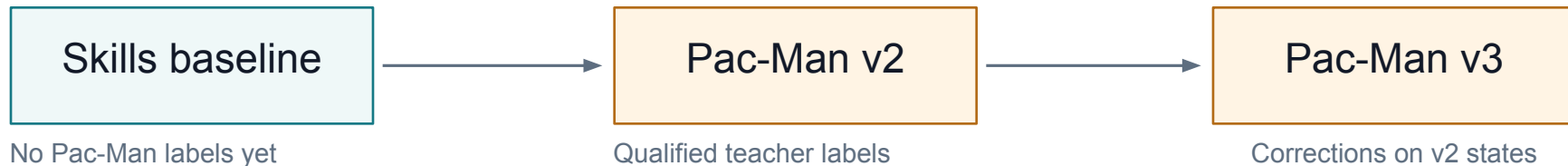
A curriculum, then task checkpoints

Each new stage loads its parent's LoRA and pointer head.

General decision curriculum — prework



Skills becomes the baseline for task adaptation



$$\varphi_{\text{child}}^0 = \varphi_{\text{parent}}$$

$$\mathbf{D}_{\text{train}} = \mathbf{D}_{\text{new}} \uplus \mathbf{D}_{\text{replay}}$$

New stage: fresh optimizer. Parent checkpoint stays saved.

Prepare Colab before class

RTX PRO 6000 Blackwell: 96 GB target

Pinned PyTorch/CUDA; BF16 autocast

Optimized FLA, convolution and fused AdamW

Pac-Man: batch 4 × accumulation 2

Checkpointing on; row budget 0

Five stages, five separate adapters

- 1 Decisions: 12,576 records $\times$ 2 epochs
- 2 Dates/evidence: 1,425 + 2,000 replay
- 3 Documents: 5,219 + 2,000 replay
- 4 Skills/devtools: 11,320 + 4,000 replay
- 5 Pac-Man: 4,096 + 2,000 replay; 1 epoch

Interactive play: meet the four ghosts

Four native ghost personalities and release rules

Remaining power pellets: 0 in the maze

After consumption: global + per-ghost frightened flags

Badge: Skills LoRA before; Pac-Man LoRA after

Planning labels from native game states

4,096 train; 256 dev; 256 evaluation

Search 20 moves, beam 8; rank survival first

1,450 training boards have power mode active

$$J = \Delta S + 5P - 4R - 6D_p + 2\min(D_g, 8)$$

J: path value; ΔS : score gain; P: pellets; R: revisits

D_p : nearest-pellet distance; D_g : distance to a dangerous ghost

CP1: fine-tune and evaluate Pac-Man

Skills → Pac-Man: rank-16 LoRA + pointer head

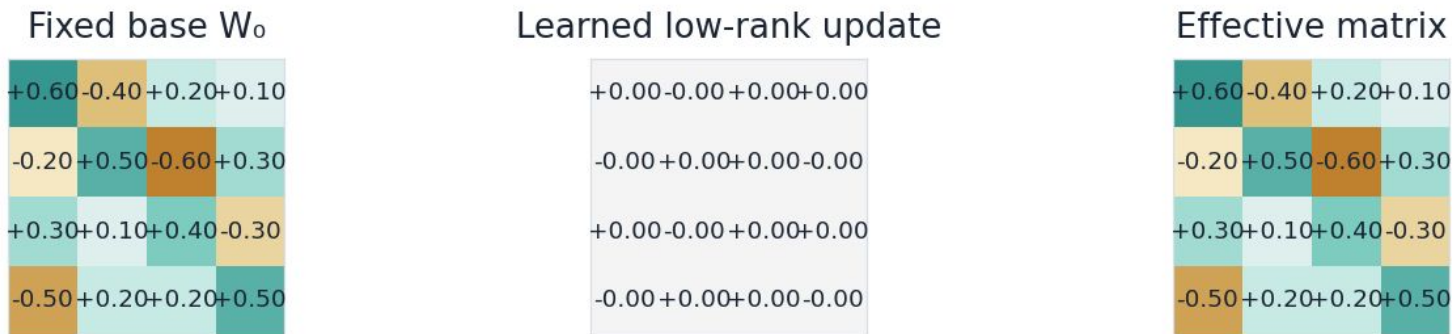
4,096 boards + 2,000 replay; one epoch

762 updates at learning rate 2e-5

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B} \sum_{i=1}^B \log p(y_i | s_i)$$

B: batch size; s_i: board; y_i: teacher move; p(y_i | s_i): predicted share

LoRA updates and training evidence



Illustrative 4×4 matrices, rank 2, $\alpha/r = 1$. Kev uses rank 16 and $\alpha = 32$.

$$W_{\text{eff}} = W_0 + \frac{\alpha}{r}BA$$

W_0 : fixed base; A : $r \times d_{\text{in}}$; B : $d_{\text{out}} \times r$

$r = 16$; $\alpha = 32$; 33.8M trainable LoRA/head parameters

[Interactive LoRA controls](#) | TensorBoard + recovery; verified Drive backup after each saved snapshot.

Evaluate decisions and gameplay

Same 256 boards: strict and tie-aware agreement

Search regret, survival choices and immediate captures

Same-seed gameplay: pellets, score and revisits

$$A = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{a}_i = y_i] \quad \hat{a}_i = \arg \max_j p(j | s_i)$$

$N = 256$; y_i : teacher move; $\hat{a}_i$: model move; $\mathbf{1}[\cdot]$: agreement indicator

Submit CP1: show what changed

Explain a changed move or remaining mistake

Connect the move to the ghost behaviour

Show trainable parameters and training curves

Save task adapter and paired evaluation results

Submit the notebook, runtime and compute use